

Take-Home Assignment: Music Catalog Insights Platform

Duration: 3 days

Type: Full-stack (Backend + Frontend + AI)

Technology Stack

- Backend: Java / Spring Boot
- Frontend: React.js / Next.js
- Database: PostgreSQL / MySQL (Any)

Problem Statement:-

Scenario

Music streaming platforms need a way to let users build and explore a personal music library, sourced from a public catalog, with analytics and AI-driven insights layered on top.

Your task: build an end-to-end web application that lets a user search a public music catalog, save selected items into their own library (your database), view analytics on that library, and get AI-generated insights from it.

Third-Party API to Integrate

Use the iTunes Search API — free, public, and requires no API key or authentication.

Docs:

<https://developer.apple.com/library/archive/documentation/AudioVideo/Conceptual/iTunesSearchAPI/>

Base endpoint: <https://itunes.apple.com/search>

You'll only need GET requests against this API.

Example Requests

- # Search albums
curl "https://itunes.apple.com/search?term=coldplay&entity=album"
- # Search songs
curl "https://itunes.apple.com/search?term=shape+of+you&entity=song"

- # Search artists
curl "https://itunes.apple.com/search?term=taylor+swift&entity=musicArtist"
- # Limit results
curl "https://itunes.apple.com/search?term=coldplay&entity=album&limit=10"
- # Lookup by Apple catalog ID
curl "https://itunes.apple.com/lookup?id=1440806041"

Sample Response Fields

```
{
  "resultCount":1,
  "results":[{
    "collectionId":1440806041,
    "artistName":"Coldplay",
    "collectionName":"Parachutes",
    "collectionPrice":9.99,
    "releaseDate":"2000-07-10T12:00:00Z",
    "trackCount":10,
    "primaryGenreName":"Alternative",
    "artworkUrl100":"https://..."
  }]
}
```

Expectation:-

Pick Your Focus (Choose ONE)

- Albums
- Songs
- Artists
- State your choice and why in README.

1. Database & Schema

- Store only user's saved library.
- Fields: id, apple_catalog_id, title, artist_name, genre, release_date, track_count/duration, artwork_url, user_rating, user_notes, created_at, updated_at.
- Use SQL or NoSQL and justify.

2. REST API Requirements

- GET /api/search?query=...&type=...
- GET /api/library
- POST /api/library
- PUT /api/library/{id}
- DELETE /api/library/{id}
- REST conventions
- Centralized error handling
- Validation
- JWT Authentication

3. Frontend / UI Requirements

- Search page
- Library page
- Analytics dashboard
- Responsive UI with loading and empty states

4. Analytics Dashboard

- Implement at least 4 charts: Bar, Pie/Donut, Line, Histogram, Horizontal Bar, Releases by Year.

5. AI Feature

- Implement ONE: Recommendations, Natural language query, Trend summary, Duplicate detection, or your own idea.

6. Deployment

- Deploy frontend and backend (Vercel, Netlify, Render, Railway, AWS, etc.).

7. Deliverables

- GitHub repo
- Live deployment
- README with setup, entity choice, schema, AI feature, trade-offs.

8. Good to Have

- Unit tests
- Pagination
- Debounced search

- Caching

Timeline Guidance

- Day 1: Backend, API proxy, DB, CRUD, JWT
- Day 2: Frontend
- Day 3: Analytics, AI, Deployment, README

Partial but well-reasoned submissions are valued over rushed, complete ones.